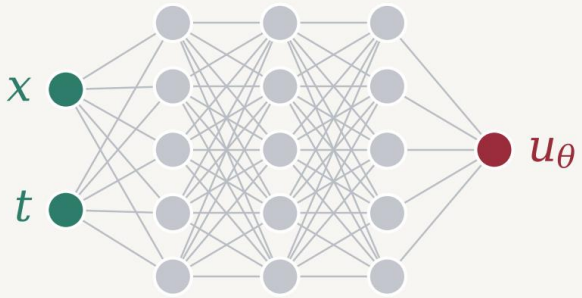
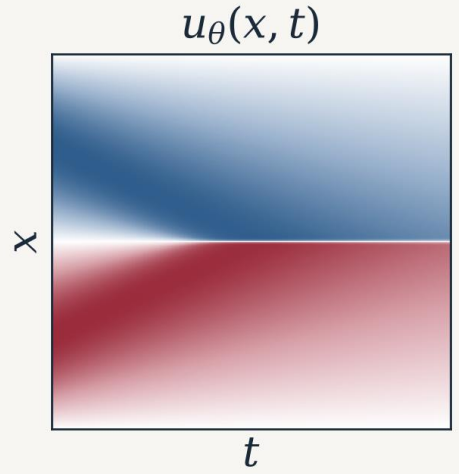
PINNS COURSE · PART 2

Physics-Informed Neural Networks





How to solve a PDE, and how to identify its coefficients

$$\partial_t u_\theta + u_\theta\,\partial_x u_\theta - \nu\,\partial_{xx} u_\theta \approx 0$$

Nassim RAZAALY

v1 · 28/10/26

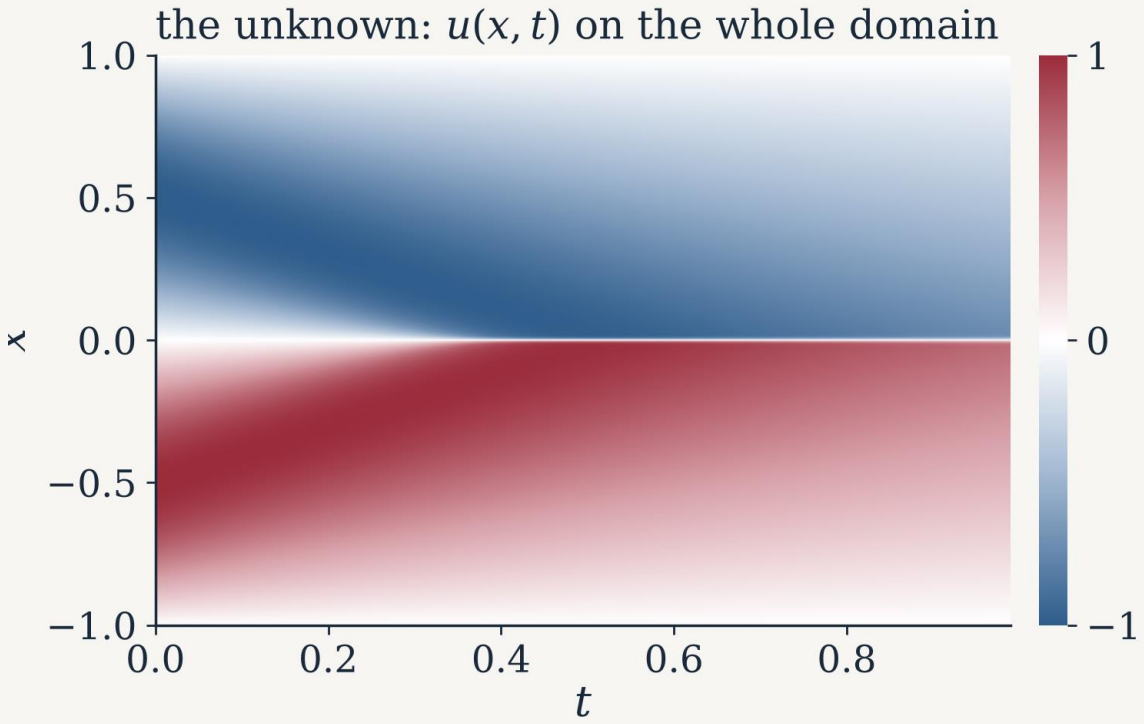
I. The forward problem: find u that satisfies a PDE

$$\partial_t u + u\,\partial_x u - \nu\,\partial_{xx} u = 0$$

with an initial and a boundary condition

$$u(x,0) = u_0(x) = -\sin(\pi x), \qquad u(\pm 1, t) = 0$$

$$x \in [-1,1], \quad t \in [0,\, 0.99], \quad \nu = 0.01/\pi$$



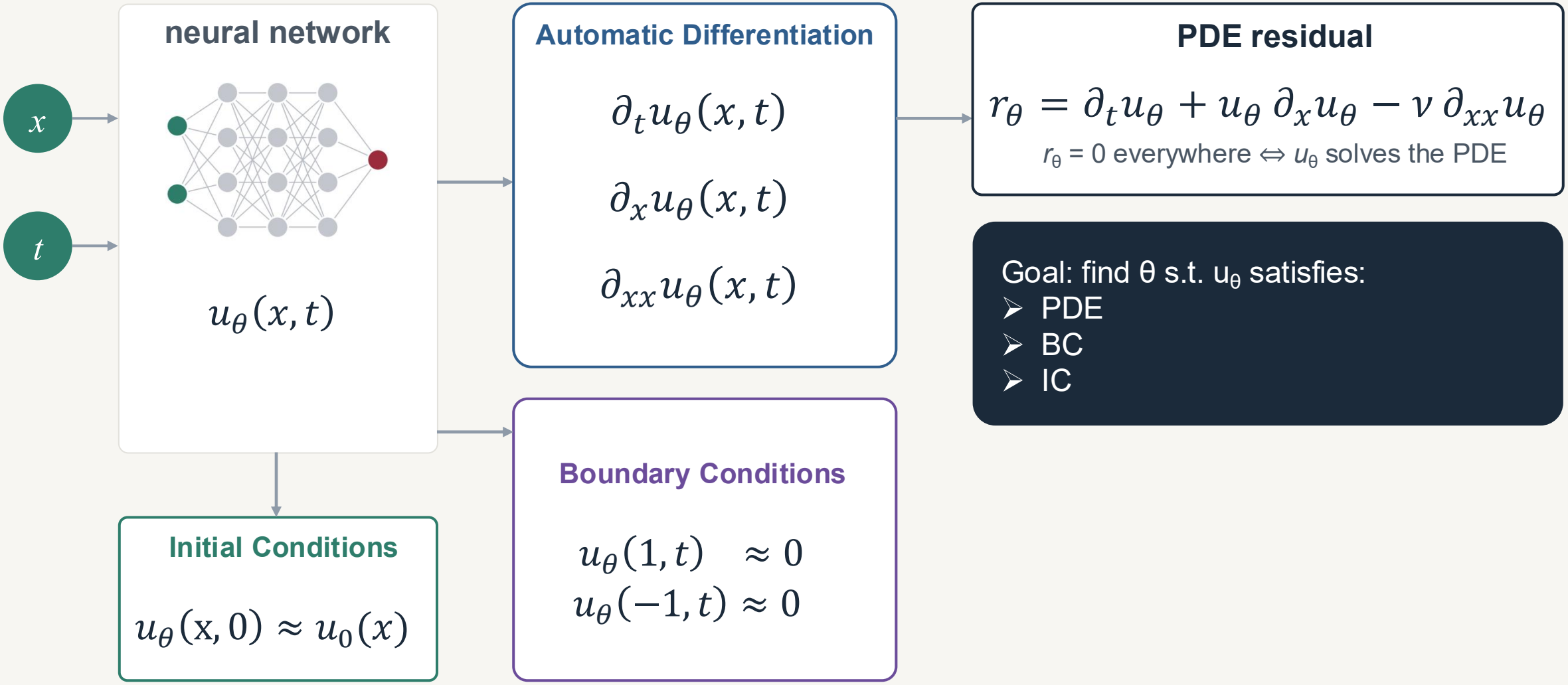
General form

$$\partial_t u + \mathcal{N}[u] = 0, \qquad \mathcal{N}[u] = u\,\partial_x u - \nu\,\partial_{xx} u$$

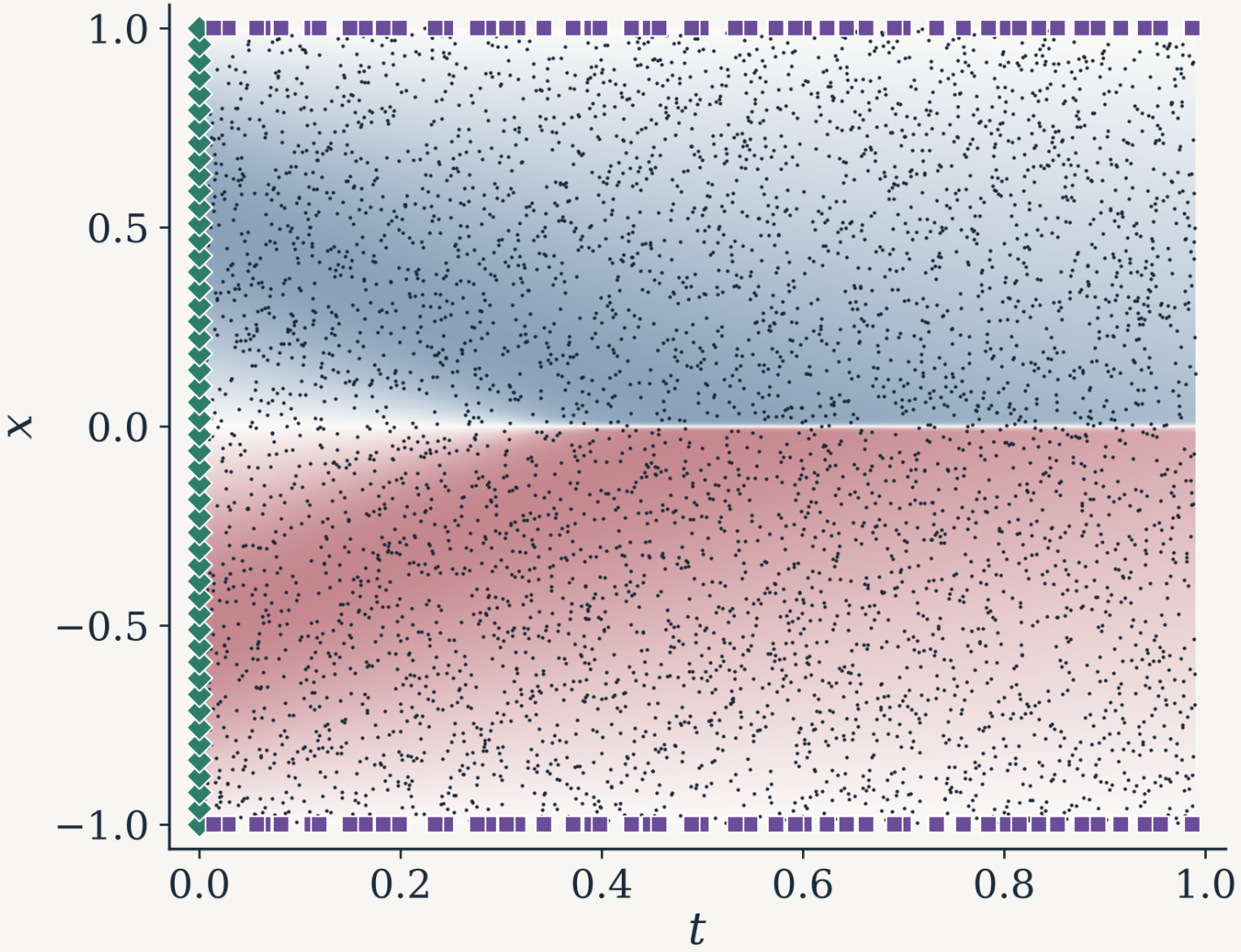
$\mathcal{N}$: any operator built from u and its derivatives

Data: the equation, u_0 and the boundary values. No value of u inside the domain.

I. The idea: replace u by a network u_θ



I. Step 1 · Collocation points: interior, boundary, initial



- **N_r interior points (x_i, t_i)**

$$r_\theta(x_i, t_i) \approx 0$$

- **$2N_b$ boundary points (± 1, t_k)**

$$u_\theta(\pm 1, t_k) \approx 0$$

- **N_0 initial points (x_j, 0)**

$$u_\theta(x_j, 0) \approx u_0(x_j)$$

Sampling: Latin Hypercube or Uniform

I. Step 2 · The loss

$$\mathcal{L}(\theta) = w_r\,\mathcal{L}_r(\theta) + w_b\,\mathcal{L}_b(\theta) + w_0\,\mathcal{L}_0(\theta)$$

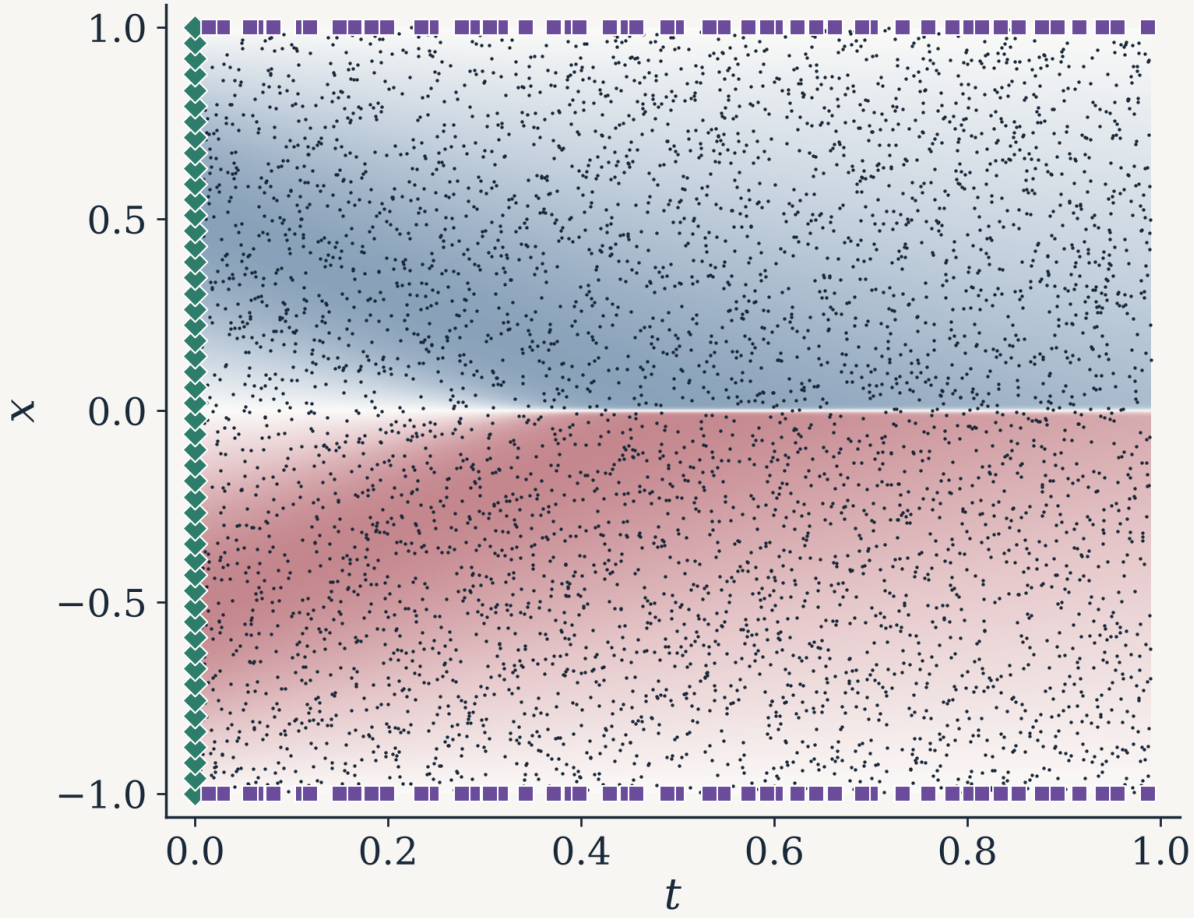
interior $$\mathcal{L}_r = \frac{1}{N_r}\sum_{i=1}^{N_r} r_\theta\,(x_i, t_i)^2$$

boundary $$\mathcal{L}_b = \frac{1}{2N_b}\sum_{k=1}^{N_b}\left(u_\theta(-1, t_k)^2 + u_\theta(1, t_k)^2\right)$$

initial $$\mathcal{L}_0 = \frac{1}{N_0}\sum_{j=1}^{N_0}\left(u_\theta(x_j, 0) - u_0(x_j)\right)^2$$

Weights w_r, w_b, $w_0 > 0$

- balance the terms; default: all equal to 1



I. Step 3 · Minimize $\mathscr{L}$: Adam first, then L-BFGS

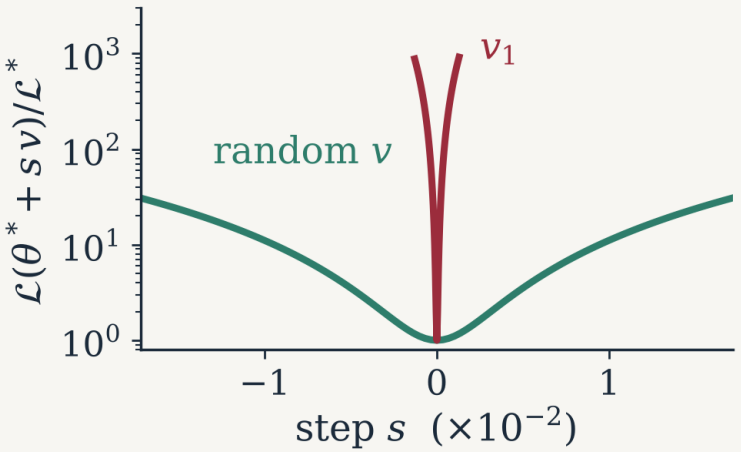
Adam · first order (gradient only)

$$\theta_{k+1} = \theta_k - \eta \frac{\hat{m}_k}{\sqrt{\hat{v}_k} + \epsilon}$$

L-BFGS · quasi-Newton (curvature)

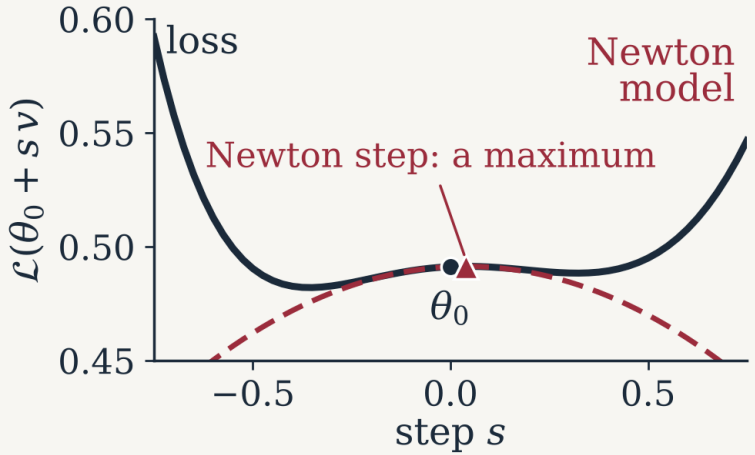
$$\theta_{k+1} = \theta_k - \eta_k H_k \nabla \mathcal{L}(\theta_k), \quad H_k \approx [\nabla^2 \mathcal{L}(\theta_k)]^{-1}$$

At the minimum θ*: a narrow valley



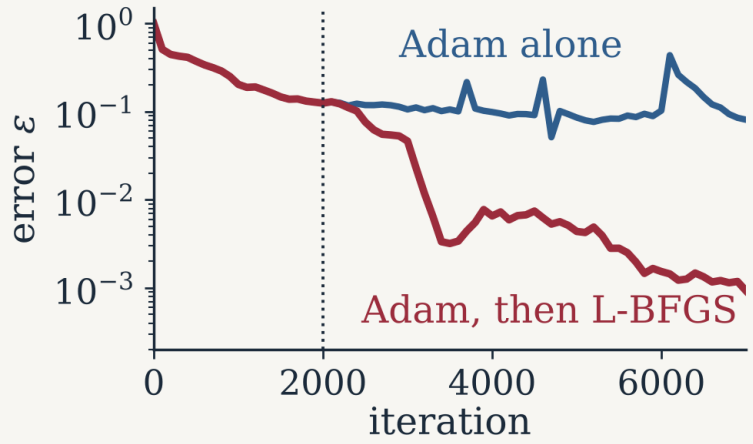
v_1: steepest direction (Hessian eigenvector)

At the random start θ_0: a hill



v: most negative curvature of $\nabla^2\mathcal{L}(\theta_0)$

Training: error ε to the exact u



same initialization, same points

- Adam: the steepest direction limits its step, so it crawls along the flat ones: alone, it stalls.
- Newton from θ_0 climbs where the curvature is negative: Adam first, then L-BFGS (H_k positive definite).

II. The inverse problem: unknown coefficients, measured u

$$\partial_t u + \lambda_1 u \, \partial_x u - \lambda_2 \, \partial_{xx} u = 0$$

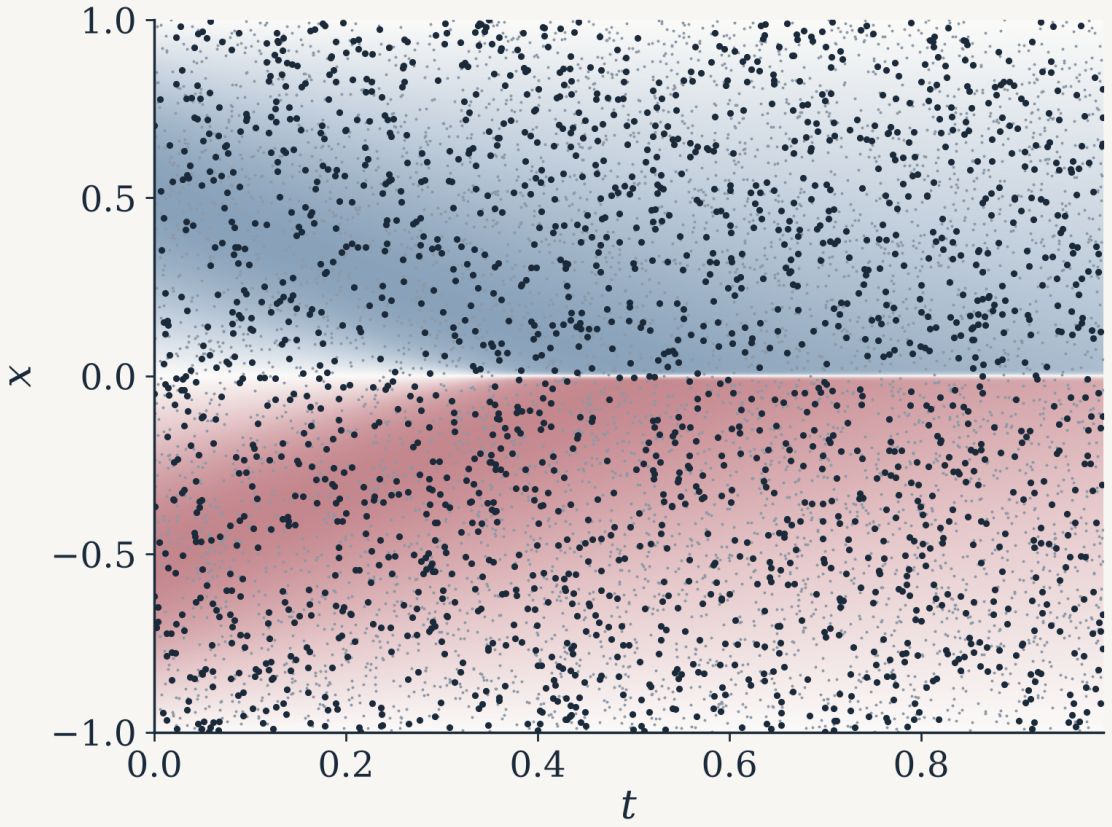
Known

- the form of the equation
- N_d measurements $u_i \approx u(x_i, t_i)$, possibly noisy

Unknown

- the coefficients λ_1, λ_2
- the solution $u(x, t)$ everywhere

No initial or boundary condition: the data replace them.



● N_d = 2000 data points: measured u_i

● N_r = 5000 interior points: no measurement

II. The loss: PDE residual and data

$$\mathcal{L}(\theta, \lambda_1, \lambda_2) = w_d \, \mathcal{L}_d + w_r \, \mathcal{L}_r$$

data $$\mathcal{L}_d = \frac{1}{N_d} \sum_{i=1}^{N_d} (u_\theta(x_i, t_i) - u_i)^2$$

interior $$\mathcal{L}_r = \frac{1}{N_r} \sum_{j=1}^{N_r} r\left(x_j, t_j\right)^2 \qquad r = \partial_t u_\theta + \lambda_1 \, u_\theta \, \partial_x u_\theta - \lambda_2 \, \partial_{xx} u_\theta$$

λ_1, λ_2 are trained with the weights!

```
lambda1 = nn.Parameter(torch.tensor(0.0), requires_grad=True)
log_lambda2 = nn.Parameter(torch.tensor(-6.0), requires_grad=True)
params = list(model.parameters()) + [lambda1, log_lambda2]

r = u_t + lambda1 * u * u_x - torch.exp(log_lambda2) * u_xx
```